STRATA

WE BUILD SAFETY, NOT ASSUMPTIONS.

Module 3 Software Documentation

Computer Vision & Motion Tracking

Active Base Stabilization and Real-Time Structural Digital Twin

CENG 396 · Group 5 · 4 November 2026

Instructor: Erkan Zergeroğlu

1 Purpose and Scope

Module 3 measures the horizontal motion of the lower base and the three building floors using a fixed external camera and printed visual markers. It converts marker positions in video frames into signed X displacements in millimetres and provides timestamped measurements to Module 4 for visualization and analysis.

This document specifies the proposed software design, including function responsibilities, inputs, and outputs. Motor control belongs to Module 1, inertial sensing to Module 2, and dashboard visualization and final comparison metrics to Module 4.

Design status and independence. Function names and interfaces below are proposed, not claims of completed implementation. Module 3 runs without live inputs from Modules 1, 2, or 4, using a test board or recorded video and a local output log.

2 Inputs and Outputs

Figure 1 summarizes the module boundary: what Module 3 receives, what it produces, and where its results are used.

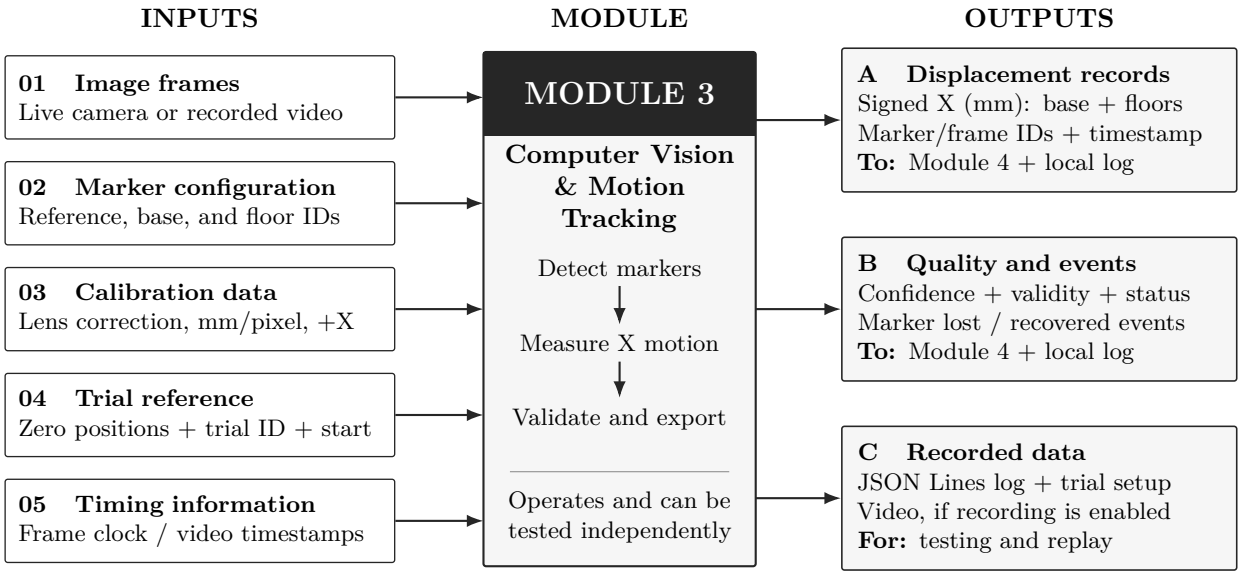


Figure 1: Module 3 input–output diagram. All inputs can be supplied locally. The same measurement records support standalone logging and later dashboard integration.

3 Software Functions: Setup and Capture

F01–F21 identify proposed Python functions. Each entry specifies its inputs, outputs, and behaviour. Setup errors stop the trial before measurement begins.

Function	Inputs, outputs, and behaviour
F01 `load_config`	**In:** JSON path. **Out:** validated `Config`. **Action:** Validate source mode/path, unique marker IDs, sign, thresholds, output paths, and enabled options; reject bad fields with `ConfigError`. Preserve settings for the trial log.
F02 `load_calibration`	**In:** File path, config. **Out:** `Calibration`. **Action:** Load camera matrix K, distortion coefficients, image size, calibration ID, and physical scale. Reject non-finite values or malformed dimensions; tracking requires positive scale.
F03 `calibrate_camera`	**In:** Board images/geometry, error limits, output path. **Out:** calibration file and fit errors. **Action:** Detect/refine board corners and solve camera parameters. Reject mixed image sizes, too few usable views, or excessive reprojection error; keep earlier valid files on failure.
F04 `estimate_scale`	**In:** Valid reference samples, known horizontal distance D, calibration/output path. **Out:** updated scale and spread. **Action:** Compute $s_k = D/\lvert u_R - u_L\rvert$; save the median mm/pixel scale and population standard deviation. Reject nonpositive distance or near-zero separation.
F05 `open_source`	**In:** Live camera index or replay path/settings. **Out:** `SourceHandle`. **Action:** Open capture and record actual image size and reported frame rate. Replay timing uses saved sidecar, valid video timestamps, or explicitly declared constant frame rate. Fail if unusable.
F06 `read_frame`	**In:** Source handle. **Out:** `FramePacket`, failure, or EOF. **Action:** Return BGR image, increasing frame ID, source time, and timing basis. Live time is monotonic host receipt time; replay preserves source timing. Reject invalid/non-increasing times; never fabricate a frame.
F07 `prepare_frame`	**In:** Frame and matching calibration. **Out:** undistorted grayscale image. **Action:** Check image size, convert to grayscale, and correct distortion using stored K. Cache correction maps; retain raw BGR image for recording. Size mismatch raises `CalibrationError`.

3.1 Configuration and Basic Types

Proposed marker map: reference IDs 0/1, BASE 10, FLOOR_1–3 IDs 21–23; dictionary `DICT_4X4_50`. `FramePacket` contains `frame_id`, uint8 BGR image $H \times W \times 3$, `source_t_ms`, and `timing_basis`. Detection corners are float arrays 4×2.

Initial software settings: 30 zeroing samples within 150 frames; loss threshold 3 frames; live read-failure limit 3; confidence threshold 0.5. Sharpness, area, stationary-noise, and calibration-error limits must be set from pilot tests and saved with the trial. These settings are proposed, not achieved accuracy or speed claims.

4 Software Functions: Tracking and Measurement

All positions below are computed in the undistorted image. Fixed references and moving targets must be approximately front-facing and in a common measurement plane. The initial stationary scene defines zero.

Function	Inputs, outputs, and behaviour
F08 `detect_markers`	**In:** Grayscale frame, dictionary/ID map. **Out:** `DetectionMap` and diagnostics. **Action:** Detect ArUco IDs/corners, refine corners, and average four corners for each centre. Exclude unknown IDs; mark duplicate expected IDs ambiguous. Detector errors return an empty map with diagnostics.
F09 `score_detection`	**In:** Image, detection, quality settings. **Out:** confidence q, usable flag, reason. **Action:** Check finite in-bounds convex corners and minimum area; evaluate sharpness and shape using the rule below. Missing/ambiguous markers have $q = 0$. A detected marker below threshold is unusable. Quality is a heuristic, not a probability.
F10 `capture_zero`	**In:** Stationary samples, calibration/config, or exact replay metadata. **Out:** `ZeroState`. **Action:** Require both references and all four targets. Save median relative centres z_i after enough complete frames; their population standard deviations must pass the stationary-noise limit. Otherwise fail zeroing. Saved zero is valid only for the same recorded trial/calibration.
F11 `resolve_reference`	**In:** Current reference detections/quality. **Out:** usable flag, u_r, confidence. **Action:** Use $u_r = (u_L + u_R)/2$ and the minimum reference confidence only if both references pass F09. Otherwise return unavailable; do not reuse a stale reference or silently fall back to one. Corrects small horizontal camera shifts only.
F12 `compute_displacements`	**In:** Detections, quality, reference, zero, scale/sign. **Out:** four target measurements. **Action:** Apply the displacement formula below. Valid confidence is the minimum target/reference score. Missing/poor targets or unusable references give null, confidence 0, and valid=false; no interpolation or held value. Displacement is relative to fixed support, not the moving base.
F13 `update_tracking`	**In:** Quality map, previous state, frame/time, threshold. **Out:** new state and events. **Action:** Count consecutive unusable detections per ID; emit loss once at the threshold and recovery once on return. Reset counts on usable detection. Reference failure alone must not increment visible target counters.
F14 `build_record`	**In:** Frame, four results, events, trial context. **Out:** validated `FrameRecord`. **Action:** Set first-frame time origin t_0 and compute elapsed time. Enforce target order, finite values, and valid/null consistency. Status precedence: reference_lost, degraded, ok. Schema failure stops the trial.
F15 `make_mock_records`	**In:** Times, amplitudes, frequency, loss windows, trial ID. **Out:** synthetic record iterator. **Action:** Generate $x_i = A_i \sin(2\pi f t)$ and configured losses using the same schema/events. Label source `cv_mock` and timing `synthetic`; reject non-finite parameters or unordered times.

$$x_i(t) = \sigma s\big[(u_i(t) - u_r(t)) - z_i\big], \qquad q = \min\left(\mathrm{clip}\left(\frac{L - S_{low}}{S_{high} - S_{low}}, 0, 1\right), r\right).$$

$\sigma = \pm 1$ is the configured X sign; s is mm/pixel; z_i is the initial relative centre. L is Laplacian variance in the marker bounding rectangle; r is shortest/longest edge length. Usability also requires geometry/area checks and the confidence threshold.

5 Software Functions: Outputs and Orchestration

The local measurement log is required. Video recording and live delivery are optional. Each trial has a unique directory; earlier files must not be overwritten.

Function	Inputs, outputs, and behaviour
F16 `start_trial`	**In:** Config, calibration, zero, actual source properties. **Out:** `TrialContext`. **Action:** Create metadata and log files; store IDs, scale, zero, thresholds, software versions, timing policy, and counters. Start with time origin unset. Resolve an enabled endpoint; resolution failure disables only publishing.
F17 `append_record`	**In:** Validated record, trial context. **Out:** write result/line count. **Action:** Persist initial time origin, then append one strict UTF-8 JSON object plus newline and flush. Copy transitions/diagnostics to the event log. Reject NaN. Required write/flush failure stops the trial as incomplete.
F18 `write_video_frame`	**In:** Raw frame, context with recording enabled. **Out:** written/disabled/failed status. **Action:** Encode raw video and save a sidecar mapping video index to frame ID/source time. Preserve zero in metadata. On recording/sidecar failure, mark video incomplete and continue measurement logging.
F19 `publish_record`	**In:** Record, endpoint/connection state, metadata. **Out:** disabled/sent/offline/failed result. **Action:** Optionally send newline-delimited JSON over TCP; send metadata on each connection. Use bounded send/connect timeouts and retry intervals. Close partial/failed connections; keep local logging and skip historical retransmission.
F20 `close_trial`	**In:** Full/partial context, source, stop reason. **Out:** `RunSummary`. **Action:** Release capture/video, close files/socket, and save counts, completion state, and stop reason when storage allows. Cleanup must be safe to repeat and must preserve the original error.
F21 `run_trial`	**In:** Configuration path and user stop signal. **Out:** summary, local files, optional live output. **Action:** Validate setup, establish zero, then execute the sequence on page 6 until EOF, stop, or fatal error. Catch per-frame source failures, maintain counters, and always call F20. No valid calibration/scale/zero means no measurement stream.

6 Data Interface and Tracking Rules

One record is produced per processed frame, with all four target entries present. Each record has `schema_version`, source/trial/frame identity, elapsed `t_ms`, timing basis, module status, measurements, and transition events.

6.1 Measurement Record

```
{
  "type": "measurement", "schema_version": 1,
  "source": "cv", "trial_id": "trial_01",
  "frame_id": 42, "t_ms": 1350.0,
  "timing_basis": "host_receipt", "status": "degraded",
  "markers": [
    {"name": "BASE", "id": 10, "x_mm": 12.4,
     "confidence": 0.95, "valid": true, "reason": "ok"},
    {"name": "FLOOR_1", "id": 21, "x_mm": 2.8,
     "confidence": 0.92, "valid": true, "reason": "ok"},
    {"name": "FLOOR_2", "id": 22, "x_mm": 4.1,
     "confidence": 0.88, "valid": true, "reason": "ok"},
    {"name": "FLOOR_3", "id": 23, "x_mm": null,
     "confidence": 0.0, "valid": false, "reason": "missing"}
  ],
  "events": [{"type": "marker_lost", "id": 23,
              "frame_id": 42, "t_ms": 1350.0}]
}
```

Numbers are illustrative, not test results. Displacement is a finite signed float in mm or null; confidence lies in [0,1]. IDs increase and timestamps follow source time. Invalid measurements always use null and confidence 0. Reasons distinguish missing, ambiguous, low-quality, reference-loss, and detection-error cases.

6.2 Loss and Recovery

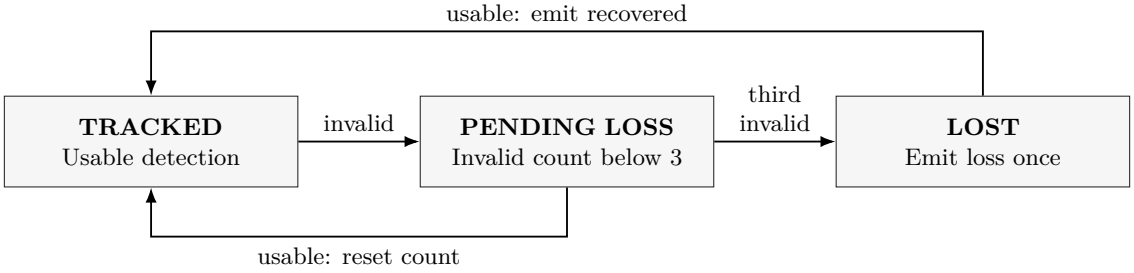


Figure 2: F13 state transitions for the proposed three-frame loss threshold. Null starts on the first invalid measurement; the event is delayed.

6.3 Time, Storage, and Delivery

Live time is host receipt time, not exposure time. Replay uses recorded sidecar timing, valid video timestamps, or a declared constant frame rate. `t_ms` is elapsed from the trial's first measurement frame. Cross-module comparison requires an agreed clock/offset; local elapsed time alone does not synchronize devices.

Save metadata and measurements in `metadata.json` and `measurements.jsonl`. Events and summaries use `events.jsonl` and `summary.json`. Optional video has a `frame_times.csv` sidecar. Metadata retains calibration, zero, sign, thresholds, timing origin, and software versions. Frameless errors use host diagnostic time and a null frame ID.

The proposed optional TCP adapter sends `trial_meta` before measurements on every connection. Receivers buffer UTF-8 until LF and discard incomplete lines on disconnect. Identity is (source, trial ID, frame ID); delivery is best effort. Proposed timeout/reconnect: 20 ms/2 s. Confirm transport with Module 4.

7 Execution, Verification, and Resources

7.1 Main Execution Sequence

```
validate config + calibration; open source          # F01,F02,F05
collect stationary samples; establish zero          # F06-F10
create trial files                                  # F16
try:
    for each source frame until EOF or user stop:    # F06
        prepare image; detect and score markers      # F07-F09
        resolve references; compute displacement     # F11-F12
        update loss/recovery; validate record        # F13-F14
        append local log                            # F17 required
        record video; publish if enabled            # F18-F19
finally:
    close resources and return summary              # F20
```

F21 owns this sequence and cleanup, including partial setup. Live read failures emit no fake frame: count consecutive failures and stop at the configured limit; reset the count after success. Bad configuration, calibration, zero, schema, or required log I/O is fatal. Target loss invalidates that target; reference loss invalidates all displacements. Dashboard or optional video failure is logged while measurement continues.

7.2 Independent Test Plan

Test	Functions	Expected result / evidence
Configuration / calibration	F01–F04	Reject duplicate IDs, bad matrices, nonpositive scale, insufficient board views, and invalid quality limits.
Known motion / stationary	F07–F12	Repeat measured 10/20 mm movements both ways; report signed error, repeatability, and stationary noise. Reject unstable/incomplete zeroing.
Common camera shift	F11–F12	Add the same horizontal shift to all synthetic coordinates; displacement is unchanged. Changed viewpoint requires recalibration.
Occlusion / reference loss	F09–F14	Null from the first invalid frame; one loss event at frame 3, none at frame 4, one recovery on return. Missing references do not falsely mark visible targets lost.
Replay / schema	F05–F07, F14, F18	Preserve source timing and stored zero; verify four targets, strict JSON, finite values, and valid/null consistency.
Output failure / cleanup	F16–F21	Log failure stops the trial; disconnected Module 4 does not. EOF, user stop, and fatal errors release resources.
Synthetic integration	F15, F19	Known waveform values and loss events reach a local reader or Module 4; records are clearly labelled `cv_mock`.

7.3 Generic Hardware and Software

Hardware: camera, fixed mount, computer, six printed markers, calibration pattern, distance reference, test board, and stable lighting.

Software: Python, OpenCV, NumPy, and standard-library utilities. Record tested versions in metadata; measure actual accuracy and processing speed.

Deliverables: program, configuration/calibration files, mock data, recorded trial/log, and test results. Confirm marker placement, X direction, time mapping, and transport with the team; standalone development uses local settings.

Technical references: OpenCV ArUco detection, camera calibration, and VideoCapture. The function names, schema, thresholds, and failure policies are the proposed project design.